实验二答辩逐句稿｜俄语口播与中文对照

现场看每段的 **动作**，完成后直接读下面的俄语。中文和俄语按同一顺序排列。先做第 1 至 4 段的网页演示，再切到 IDEA 和 Helios 结果。第 3 段的字段校验时间紧时可以省略，最后的代码和问答供老师追问时翻看。

实验原题	我在哪一步展示完成情况
第 1 项，网页批量导入、嵌套对象、输入校验、整批回滚	第 1 至 3 段
第 2 项，按角色查看历史、四个必需字段	第 4 段
第 3 项，在业务层检查新唯一性规则	第 2、5 段
第 4 项，JMeter 覆盖创建、编辑、删除和导入，并解释隔离策略	第 5、6 段

答辩前准备好 A、B 和管理员三个独立的浏览器会话。手动上传用同一组 `01-success.json` 和 `02-duplicate-last.json`，先传 01 再传 02。可直接使用现成文件，已经上传过 01 就用文末 PowerShell 生成新的一组。 实验原题 · Helios 网页 · 上传文件与搜索名

0. 开场

动作　停在应用首页，指左侧 **Импорт JSON** 和 **История импорта**。

俄语直接读

> Во второй лабораторной работе я расширил систему из первой работы. Я добавил загрузку нескольких фильмов из файла, историю импорта с правами доступа и проверку уникальности фильма по названию и режиссёру. После демонстрации в браузере я покажу, как проверил одновременную работу пользователей на сервере Helios.

中文对照

> 我在实验一的系统上增加了从文件批量导入电影、按权限查看导入历史，以及检查电影名称和导演是否重复的功能。网页演示之后，我会展示如何在 Helios 服务器上验证多个用户同时操作。

1. 第 1 项｜从网页上传、批量导入和嵌套对象

动作 1　打开本次要上传的 `01-success.json`，指出 `movies` 数组里的两条记录，以及第一条里的 `coordinates`、`director` 和 `director.location`。

俄语直接读

> В массиве movies два фильма. Координаты и данные режиссёра находятся внутри записи фильма. Местоположение вложено в данные режиссёра. Так в одной записи передаются и фильм, и его вложенные объекты.

中文对照

> movies 数组里有两部电影，坐标和导演写在电影记录内，地点写在导演对象内。这样一条记录就带齐了电影和它的嵌套对象。

动作 2　用普通用户 A 打开 **Импорт JSON → Выбрать файл → Загрузить**。上传 01，记下操作 ID。等页面显示新增 2 部再读。

IDEA 定位　读到「браузер отправляет」时，指 `import-form` 提交回调中的 `form.append('file', …)` 和 `api('imports', 'POST', form)`。 app.js 第 805 行
读到「сервер передаёт」时，指 `upload()` 中的 `imports.upload(...)`。 ImportResource.java 第 49 行
读到同一个护照号时，指 `person()` 里的 `cache.get(input.passportID)`、`p.passportID=:passport` 和 `samePerson(...)`。 ImportService.java 第 136 行

俄语直接读

> Я загружаю этот файл через веб-страницу. Браузер отправляет его в поле file, а сервер передаёт содержимое в сервис импорта. На экране появился ID операции и результат: добавлено два фильма. У обоих фильмов одинаковый номер паспорта режиссёра, и его данные совпадают. Сервис находит одного и того же человека, поэтому обе записи ссылаются на одного режиссёра.

中文对照

> 我通过网页上传文件。浏览器把文件放在名为 file 的字段中，服务器再把文件内容交给导入服务。现在页面显示了操作 ID 和新增两部电影。两部电影填的是同一个导演护照号，资料也一致，因此服务复用同一个人物。

2. 第 1、3 项｜第二条撞重，证明整批回滚

动作 1　打开本次要上传的 `02-duplicate-last.json`，指第一条带 `-rollback` 的新电影和新导演。再对比 01，指出两份文件的第二条完全相同。用 A 上传 02，记下失败的操作 ID。

俄语直接读

> Теперь я проверю требование об откате всего импорта при ошибке. В этом файле первый фильм новый, и у него новый режиссёр. Второй фильм совпадает по названию и режиссёру со вторым фильмом из уже загруженного файла. Сервер вернул конфликт дубликата на поле режиссёра второго фильма.

中文对照

> 现在验证导入遇到错误时整批回滚。这个文件的第一部电影和导演都是新的，第二部电影的名称和导演与刚才成功导入的第二部相同。服务器报告第二条的导演字段发生重复冲突。路径里的下标 1 代表第二条，因为下标从 0 开始。

动作 2　在 **Фильмы** 用 **Колонка → Название → Точное значение → Включить фильтр → Применить** 查第一条 `-rollback`，再查 01 中已经提交的第二条。 样例的完整搜索名

IDEA 定位，按口播顺序切换

读到的俄语	打开哪个文件，指哪一段
«В коде вызов db.write стоит снаружи цикла»	在 `upload()` 指第 68 行的 `return db.write(em -> {`，再指第 72 行开始的 `for` 循环。 ImportService.java 第 68 行
«flush отправляет изменения в базу»	在 `saveImported()` 指第 195 行的 `em.persist(movie)` 和第 196 行的 `em.flush()`。 MovieService.java 第 192 行
«фиксация происходит после обработки всего файла»	在 `once()` 指第 82 行的 `work.apply(em)` 和后面的第 85 行 `tx.commit()`。 Database.java 第 82 行
«возникает исключение, и транзакция откатывает»	先指 `upload()` 在第 83 行把带记录路径的异常抛出，再指 `once()` 捕获异常后第 90 行的 `tx.rollback()`。 ImportService.java 第 83 行 · Database.java 第 90 行
«Запись об ошибке сохраняется … отдельной транзакцией»	回到 `upload()` 的失败分支，指第 109 行独立调用的 `db.metadata(...)` 和第 112 行的 `op.status = ImportStatus.FAILED`。 ImportService.java 第 109 行

附录 B 和 E 贴有对应代码。

俄语直接读

> Поиск не находит первый фильм из отклонённого файла. При этом фильм из предыдущей, успешной загрузки остался в базе. В коде вызов db.write стоит снаружи цикла, поэтому весь файл записывается в одной транзакции. При обработке первого фильма flush отправляет изменения в базу, а фиксация происходит после обработки всего файла. На втором фильме возникает исключение, и транзакция откатывает фильм, координаты и новые связанные объекты из этого файла. Запись об ошибке сохраняется в истории отдельной транзакцией.

中文对照

> 搜不到失败文件的第一部电影，但上一次成功导入的电影还在。代码里 db.write 在循环外，所以整个文件共用一次写事务。第一条执行 flush 时，修改已经同步到数据库，但要处理完整个文件才提交。第二条抛异常时，事务回滚这个文件里的电影、坐标和新建的关联对象。错误记录由单独的事务保存到历史中。

3. 第 1 项｜沿用实验一的输入约束（可选）

动作　打开样例，指出第二条记录里的 `"length": 0`。用 A 上传，看到 `INVALID_FIELD` 和 `movies[1].length` 后再读。 03-invalid-length.json

IDEA 定位　先指 `length` 字段上面的 `@Positive`。再指 `object()` 里的 `MovieService.validate(result)` 和 `throw p.at(row, path + "." + p.path)`。讲「写事务之前」时，指 `upload()` 里第 67 行的解析和紧接的第 68 行 `db.write`。 Movie.java 第 52 行 · ImportParser.java 第 85 行 · ImportService.java 第 67 行

俄语直接读

> В первом задании также требуется проверять поля по правилам первой лабораторной работы. Например, продолжительность фильма должна быть больше нуля. Я поставил ноль во второй записи. Сервер указал точное поле ошибки. Этот файл отклоняется ещё при разборе и проверке данных, до начала транзакции записи фильмов.

中文对照

> 第一项还要求按实验一的规则校验输入。比如电影时长必须大于零。我把第二条的时长设为零，服务器指出了具体的错误字段。这个文件在解析和验证时就被拒绝，早于电影写事务开始。

4. 第 2 项｜逐列展示历史字段，再切换账号

动作 1　打开 A 的 **История импорта**。指着 01 和 02 的两条记录，按 **ID** → **Статус** → **Пользователь** → **Добавлено** 的顺序移动鼠标。上传了 03 时，用文件名或操作 ID 找准 02。

俄语直接读

> Теперь второе требование — история импорта. Здесь есть все четыре поля из задания. В первой колонке — ID операции. В колонке «Статус» видно, чем она завершилась. В колонке «Пользователь» указан тот, кто запустил импорт. В колонке «Добавлено» количество показывается только при успехе. В строке со статусом «Успешно» показано число два, а в строке со статусом «Ошибка» стоит прочерк. Дополнительно я показываю имя файла, время начала и причину ошибки.

中文对照

> 现在展示第二项要求，导入历史。表中有题目规定的四个字段。第一列是操作 ID，第二列是完成状态，第三列是发起导入的用户，第四列是新增数量。只有成功时显示数量。这里状态为「Успешно」的一行显示 2，状态为「Ошибка」的一行显示横线。表里还显示文件名、开始时间和失败原因。

动作 2　切到已登录的普通用户 B，查看相同的操作 ID。再切到管理员，找回这两条记录。

IDEA 定位

- 讲「列表按所有者筛选」时，在 `list()` 指 `actor.role == UserRole.ADMIN ? "" : " WHERE o.user.id=:user"` 。讲按 ID 查权限时，在 `get()` 指第 65 行的所有者检查。 ImportHistoryService.java 第 50 行 · ImportHistoryService.java 第 65 行
- 讲「只在成功时显示数量」时，指 `dto()` 的 `row.put("addedCount", …)` ，再指 `refreshHistory()` 里失败行填 `'–'` 的条件。 ImportHistoryService.java 第 36 行 · app.js 第 747 行
- 讲「历史自动更新」时，在 `setInterval()` 指直接调用 `refreshHistory()` 的分支。若老师问四列在哪里定义，打开 HTML 的历史表头。 app.js 第 703 行 · index.html 第 280 行

俄语直接读

> Обычный пользователь видит только запущенные им операции. Я переключаюсь с пользователя A на пользователя B. Здесь двух операций пользователя A нет. Теперь открываю историю администратора. В ней видны обе операции, и в колонке «Пользователь» по-прежнему указан A. Сервер применяет фильтр по владельцу и проверяет доступ при запросе отдельной операции по её ID. Он передаёт число

> добавленных фильмов только при статусе SUCCESS. Страница истории запрашивает обновления автоматически.

中文对照

> 普通用户只能看到自己发起的操作。切换到 B，A 的两条记录看不到。再打开管理员的历史，这两条都能看到，发起人仍然是 A。服务器查询列表时按所有者过滤，直接按 ID 查一条操作时也检查权限。后端只在状态为 SUCCESS 时返回新增数量。历史页会自动请求更新。

5. 第 3、4 项｜在 IDEA 讲组合唯一性、写锁和版本

动作 1｜IDEA 定位 打开 `check()`，依次指第 14 行构造 `MovieKey`、第 15 行 `IS NULL`、第 19 行排除当前电影 ID 和第 22 行抛出 `DUPLICATE_MOVIE`。如果要展示共用校验器，打开普通保存和导入的方法。MovieUniquenessValidator.java 第 14 行 · MovieService.java 第 117 行 · MovieService.java 第 192 行

俄语直接读

> По третьему требованию я согласовал с преподавателем новое правило уникальности. Ключ состоит из точного названия фильма и ID режиссёра. Если у режиссёров одинаковые имена, но разные ID, ключи различаются. Если у двух фильмов одно название и у обоих нет режиссёра, ключи совпадают. При редактировании проверка исключает текущий фильм по его ID. Создание, редактирование и импорт вызывают один и тот же валидатор. Это правило проверяет сервис приложения. Для этой пары в базе нет ограничения UNIQUE.

中文对照

> 对于第三项，我和老师确认了新的唯一性规则。组合键由电影的精确名称和导演 ID 组成。导演同名但 ID 不同，组合键就不同。同名电影都没有导演时，组合键相同。编辑时按 ID 排除正在编辑的电影。新增、编辑和导入共用同一个校验器。这条规则由应用服务检查，数据库没有给这对字段加 UNIQUE 约束。

动作 2｜IDEA 定位 打开 `once()`，指第 76 行设置隔离级别、第 80 行给 `AppState` 加 `PESSIMISTIC_WRITE`、第 82 行执行 `work.apply(em)` 和第 85 行提交。讲 Helios 实际用的级别时，指 `configuration.json` 中的 `isolation`。 Database.java 第 76 行 · Helios 运行配置

俄语直接读

> Если два пользователя одновременно создают фильм с одним ключом, оба запроса могли бы сначала не найти дубль. Поэтому транзакция записи сначала блокирует общую строку AppState. Проверка и запись выполняются после получения блокировки. Второй запрос ждёт. Когда первый фиксирует изменения, второй начинает проверку и при READ

> COMMITTED уже видит сохранённый фильм. Я оставил этот уровень изоляции, потому что для проверяемых операций запись уже упорядочена общей блокировкой. Это подтвердили тесты на Helios.

中文对照

> 两个用户同时创建同一个键时，如果各自先查，可能都查不到重复。因此写事务先锁住 AppState 的公共行，拿到锁才查重和写入。第二个请求等待。第一个提交后，第二个才开始查询，在 READ COMMITTED 下能看到刚提交的电影。这里保留这个隔离级别，因为要验证的写操作已经由公共锁排队，Helios 测试也验证了这一点。

动作 3｜IDEA 定位 先在 `saveInTransaction()` 指第 126 行读取表单的 `version`，第 127 行和当前值比较。再在 `delete()` 指第 202 行的 `required(...)` 和第 204 行的版本比较。解释版本由谁更新时，指电影实体上的 `@Version`。 MovieService.java 第 126 行 · MovieService.java 第 202 行 · Movie.java 第 15 行

俄语直接读

> Ещё я проверяю версию фильма. Пользователь мог открыть форму до чужого изменения и сохранить её позже. Сервер сравнивает версию из формы с текущей версией в базе. Если первым прошло редактирование, удаление со старой версией получает конфликт. Если первым прошло удаление, при редактировании сервер уже не находит фильм.

中文对照

> 我还会检查电影版本。用户可能在别人修改前就打开了表单，很晚才保存。服务端拿表单里的版本和数据库当前版本比较。编辑先成功时，使用旧版本的删除会冲突。删除先成功时，后到的编辑找不到电影。

6. 第 4 项｜展示 JMeter 怎样模拟并发，Helios 怎样验结果

动作 0（老师要求现场启动时） 用 Windows PowerShell 登录 Helios，按运行说明用 `start-test` 启动并准备账号。正式网页和测试实例共用端口，先做完前面的网页演示再切换。现场只跑 **C03 两轮**，运行结束后依次展示新目录的 `summary.json` 和 `C03/responses.jsonl`。完整验收的 20 轮数据用下面动作 2 中的归档。 Helios 完整启动流程

俄语直接读

> Сейчас я запускаю на Helios короткую проверку C03. Для этого я запускаю тестовый экземпляр командой start-test и подготавливаю тестовые учётные записи. Затем скрипт

> запускает JMeter в командной строке. Я показываю два раунда, а результаты полного прогона за двадцать раундов уже сохранены отдельно.

中文对照

> 现在我在 Helios 上跑一次 C03 短测。先用 start-test 启动测试实例，再准备测试账号。然后脚本用命令行启动 JMeter。现场展示两轮，完整的 20 轮结果已单独归档。

动作 1｜直接展示 JMX 的 XML 打开 `lab2-concurrency.jmx` 的源码，按下表的顺序指给老师看。附录 F 贴了原始 XML 和脚本对应关系。 JMeter 测试计划 XML

口播到这里	在 JMX 指哪一行
«один план для трёх сценариев»	第 12–18 行的 `scenario` 、 `threads` 和外层 `loops=1` ，再指第 81 行的内层 `rounds` 。C02 实际线程数看 `command.json` 的 `-Jthreads=10` 。C02 实际命令
«каждый поток входит в систему один раз»	第 23–25 行的 `CookieManager` ，第 43–44 行的 `POST /auth` ，再指第 73 行的 `login.groovy` 。
«встречаются на барьере»	第 135 行的 `barrier.groovy` 和第 142–143 行的 `groupSize` 、 `timeoutInMs` 。
«передаёт JSON в поле file»	先指第 148、198 行的两个条件，再指第 209–218 行的 `POST /imports` 、 `DO_MULTIPART_POST` 、 `File.path` 和 `File.paramname` 。
«проверяет ожидаемый результат»	指两条目标请求下的 `record.groovy` ，在第 189、239 行。

俄语直接读

> Здесь один план для трёх сценариев. Команда запуска передаёт номер сценария, число потоков и число раундов. В полном прогоне в C01 и C03 по два потока, в C02 — десять, и везде по двадцать раундов. Каждый поток входит в систему один раз и сохраняет свой сеанс. В каждом раунде поток берёт своё действие из подготовленных данных. Перед отправкой запросов потоки встречаются на барьере, чтобы начать почти одновременно. В C01 и C02 метод и путь берутся из данных сценария. В C03 каждый поток передаёт JSON в поле file. После запроса один и тот же скрипт проверяет ожидаемый результат и записывает фактический ответ.

中文对照

> 一份测试计划用于三个场景，启动命令传入场景编号、线程数和轮数。完整测试中 C01、C03 各有 2 个线程，C02 有 10 个，每组 20 轮。每个线程登录一次并保留自己的会话。每轮从预先准备的数据中取出自己的操作，线程在屏障处会合，让请求尽量同时发出。C01、C02 从场景数据中读取 HTTP 方法和路径，C03 用 file 字段上传 JSON。请求结束后，同一个脚本检查响应是否符合预期，并保存实际响应。

动作 2｜指着结果讲，约 1 分钟　打开 Helios 的 `summary.json`。现场跑了完整测试时，在同一个 Helios 终端用 `nl -ba "$run_dir/summary.json"` 带行号显示新结果。下面的行号和数字取自归档。展示新结果时按字段名找，并用当次的 `editWins` 和 `deleteWins`。 Helios 结果汇总

开头　先指第 3、29、53 行的三个 `scenario`，再指每组的 `rounds: 20`，在第 5、31、55 行。

> Я запустил на Helios три сценария JMeter, по двадцать раундов каждый.

中文对照：我在 Helios 上运行了三个 JMeter 场景，每组二十轮。

C01　指第 4 行的 `threads: 2`，再指第 7 至 9 行的 `requests: 40`、`commits: 20`、`expectedConflicts: 20`。最后指第 22、23 行的 `editWins: 9` 和 `deleteWins: 11`。

> В C01 два пользователя одновременно редактируют и удаляют один фильм. В каждом раунде проходит только одна операция. В этом прогоне девять раз победило редактирование, одиннадцать раз — удаление. Тест проверяет новый слоган и версию либо отсутствие фильма.

中文对照：C01 中两个用户同时编辑和删除同一部电影，每轮只有一个操作成功。这次编辑先成功 9 次，删除先成功 11 次。测试会核对新标语和版本，或者确认电影已被删除。

C02　指第 30 行的 `threads: 10`。再指第 33 至 37 行的 `requests: 200`、`commits: 20`、`expectedConflicts: 180` 和 `duplicates: 0`。

> В C02 десять потоков одновременно создают фильм с одинаковым названием и одним ID режиссёра. В каждом раунде сохраняется только один фильм. За двадцать раундов получилось двадцать успешных созданий и сто восемьдесят ожидаемых конфликтов. Дубликатов в базе нет.

中文对照：C02 中 10 个线程同时创建同名、同导演 ID 的电影。每轮只保存一部。20 轮一共创建成功 20 次，另有 180 次预期冲突，数据库里没有重复项。

C03 指第 57 至 59 行的 `requests: 40` 、 `commits: 20` 和 `expectedConflicts: 20` 。然后指第 72、73 行的 `lateDuplicateConflicts: 20` 和 `failedImportResidue: 0` 。

> В C03 два файла загружаются одновременно. Один файл импортируется целиком, другой получает конфликт на второй записи и откатывается целиком. Здесь видно двадцать успешных импортов и двадцать конфликтов на второй записи. После отката лишних записей в базе нет.

中文对照：C03 中两个文件同时上传，一份整批成功，另一份在第二条发生冲突并整批回滚。这次有 20 次成功导入和 20 次第二条撞重。失败文件没有在数据库里留下记录。

说明预期冲突 指第 10、36、60 行的 `unexpected: 0` ，再指第 20、46、70 行的 `passed: true` 。只有当次三组结果都是这样时才读这段。

> Во всех трёх сценариях неожиданных результатов нет. Тест проверяет и ответы сервера, и итоговое состояние базы. Ожидаемый конфликт засчитывается как успешная проверка.

中文对照：三个场景都没有意外结果。测试检查服务器响应和数据库最终状态。符合预期的冲突也算一次通过的检查。

IDEA 定位 老师问用例怎么构造时，打开 `prepare()` 。C01 指第 69 至 75 行共用版本的 `PUT` 和 `DELETE` 。C02 指第 78 至 80 行所有线程复用的创建请求。C03 指第 83 至 85 行两个文件的电影名和护照号。 lab2-jmeter.py 第 67 行

动作 3｜老师追问 C03 回滚时 打开归档的 `C03/responses.jsonl` ，指第 1 行的 `409` 、 `movies[1].director` 、 `passed=true` ，再指第 2 行同一轮的 `201` 。接着打开 `C03/assertions.json` ，指第 14 至 17 行的四类数据增量和第 22 行的 `failedImportResidue: 0` 。展示现场新结果时，找同一轮的 `409` 和 `201` ，按实际结果说哪一个线程获胜。 C03 原始响应 · C03 数据库断言

IDEA 定位 读到「409 符合预期」时，指 `record.groovy` 第 19 至 25 行的状态码、错误码、字段路径和 `prev.setSuccessful(valid)` 。读到「只保留获胜文件」时，打开 `verify()` ，指第 158 行的 `Counter` 比较和第 166 行开始的 `domain_delta` 检查。 record.groovy 第 19 行 · lab2-jmeter.py 第 158 行 · lab2-jmeter.py 第 166 行

俄语直接读

> Вот первая пара ответов с Helios. Файл участника ноль получил код четыреста девять на второй записи. Файл участника один получил код двести один. В этой проверке четыреста девять — ожидаемый ответ, поэтому у запроса стоит passed true. После каждого раунда скрипт сверяет названия оставшихся фильмов с файлом победителя. По снимкам базы за

> все двадцать раундов добавилось сорок фильмов, сорок записей координат, двадцать человек и двадцать местоположений. Эти числа соответствуют только успешным файлам.

中文对照

> 这是 Helios 保存的第一对响应。线程 0 的文件在第二条收到 409，线程 1 的文件收到 201。这里 409 符合预期，所以该请求的断言通过。每轮结束后，脚本把留下的电影名和获胜文件逐项比较。20 轮前后的数据库快照显示新增 40 部电影、40 条坐标、20 个人物和 20 个地点，数量与成功文件完全相符。

7. 收尾

动作　停在 Helios 结果汇总。

俄语直接读

> Всего в трёх сценариях было двести восемьдесят целевых запросов. Шестьдесят операций прошли успешно, двести двадцать получили ожидаемый отказ. Неожиданных результатов не было. Во всех раундах проверено итоговое состояние данных.

中文对照

> 三组场景共发出 280 次目标请求。60 次成功，220 次按预期被拒绝，没有意外结果。测试检查了每轮结束后的数据状态。

老师追问时直接读

一次导入到底用了几个事务?

IDEA 定位　在 `upload()` 依次指第 58 行的 `db.metadata`、第 67 至 68 行的预检和主事务，以及第 109 行保存失败状态的 `db.metadata`。 ImportService.java 第 58 行 · ImportService.java 第 109 行

俄语

> При успешном импорте выполняются две транзакции. Первая сохраняет запись со статусом RUNNING, а вторая фиксирует фильмы и статус SUCCESS. Если ошибка возникает во время записи фильмов, вторая транзакция откатывается, а третья сохраняет FAILED. При ошибке предварительной проверки транзакция записи фильмов ещё не началась.

中文

> 成功导入用两个事务。第一个保存 RUNNING，第二个一起提交电影和 SUCCESS。如果错误发生在写电影时，第二个事务回滚，第三个保存 FAILED。如果在预检时就失败，电影写事务还没有开始。

业务逻辑层和数据库层分别负责什么?

IDEA 定位 按请求调用顺序打开 `ImportResource.upload()`、`ImportService.upload()`、`Database.once()`。 ImportResource.java 第 31 行 · ImportService.java 第 51 行 · Database.java 第 71 行

俄语

> Слой web принимает HTTP-запросы. Слой service определяет, какие данные проверять и какие действия объединять в транзакцию. Слой persistence открывает транзакцию, задаёт изоляцию и вызывает commit или rollback. База данных выполняет запросы и обеспечивает действие блокировок и транзакций.

中文

> web 层接收 HTTP 请求。service 层决定校验哪些数据、哪些操作要放在同一事务里。persistence 层开启事务、设置隔离级别，再调用提交或回滚。数据库负责执行查询，并落实锁与事务。

这个项目用 JTA 或声明式事务吗?

IDEA 定位 指 Helios 持久化单元的 `transaction-type="RESOURCE_LOCAL"`、`once()` 里的 `EntityTransaction` 和服务类上的 `@ApplicationScoped`。 persistence.xml 第 21 行 · Database.java 第 73 行 · ImportService.java 第 14 行

俄语

> Для доступа к одному PostgreSQL я использую локальные транзакции JPA в режиме RESOURCE_LOCAL. В классе Database границы задаются программно через EntityTransaction. Сервисы — компоненты CDI с областью ApplicationScoped. При декларативном управлении границы задаются аннотацией, а транзакцией управляет контейнер.

中文

> 项目访问一个 PostgreSQL，使用 JPA 的 RESOURCE_LOCAL 本地事务。Database 用 EntityTransaction 编程式管理事务边界。服务是 ApplicationScoped 的 CDI 组件。声明式管理则由注解定义规则，由容器管理事务。

实验中的 ACID 怎么对应?

IDEA 定位 原子性指 `db.write` 包住循环。一致性指解析校验和业务键校验。隔离性指行锁和版本比较。持久性指 `tx.commit()`。 ImportService.java 第 68 行 · ImportParser.java 第 86 行 · MovieUniquenessValidator.java 第 14 行 · Database.java 第 80 行 · MovieService.java 第 126 行 · Database.java 第 85 行

俄语

> Атомарность означает, что файл записывается целиком или откатывается целиком. Согласованность поддерживают проверка полей и проверка бизнес-ключа. Изоляцию записи обеспечивают READ COMMITTED, блокировка AppState и контроль версий. Долговечность означает, что после commit данные сохраняются в PostgreSQL.

中文

> 原子性体现在整个文件一起提交或回滚。一致性由字段校验和业务键校验维护。写入隔离依靠 READ COMMITTED、AppState 行锁和版本检查。持久性体现在提交后的数据保存在 PostgreSQL 里。

老师问到 EJB、JTA 或 Spring 时怎么说?

俄语

> В Jakarta EE Stateless Bean не хранит состояние разговора с клиентом, Stateful Bean хранит его между вызовами, Singleton Bean существует в одном экземпляре на приложение, а Message-Driven Bean обрабатывает сообщения. В моём проекте сервисы реализованы через CDI. JTA позволяет контейнеру координировать транзакцию и при необходимости подключать несколько ресурсов. В Spring декларативную транзакцию можно задать через Transactional, а программную — через TransactionTemplate. Здесь границы задаются явно через EntityTransaction.

中文

> Jakarta EE 中，Stateless Bean 不保留客户端会话状态，Stateful Bean 能在多次调用之间保留状态，Singleton Bean 在一个应用内只有一个实例，Message-Driven Bean 用于处理消息。本项目的服务用 CDI 实现。JTA 可以由容器协调事务，必要时让多个资源参与。Spring 可以用 Transactional 声明事务，也可以用 TransactionTemplate 编程式管理。本项目通过 EntityTransaction 显式设置边界。

Session Bean 生命周期和其它理论题见 实验二答辩要点。

答辩前准备

1. 在 Helios 的 `/home/studs/s407960/is-lab2-4101` 运行 `python3.11 scripts/helios.py status` 。在 Windows 另开终端建立隧道。 启动说明
2. 用独立会话登录 A、B 和管理员。打开 `lab2-concurrency.jmx` 的 XML 源码，并准备结果汇总和 C03 原始结果。 JMeter 配置 · Helios 汇总
3. 首次使用可以直接选下表文件。重复演示时，在 Windows PowerShell 执行下方生成命令，使用它打印的 `UPLOAD 01` 、 `UPLOAD 02` 和搜索名称。两份必须来自同一次生成。 手动测试说明

网页动作	文件
成功导入，先上传	01-success.json
验证末条撞重，再上传	02-duplicate-last.json
字段校验，可单独上传	03-invalid-length.json

```
ssh -N -L 18083:127.0.0.1:40797 -o ExitOnForwardFailure=yes -p 2222 s407960@helios.cs.ifmo.ru
```

重新生成 01、02 的命令。新文件放在 `%TEMP%\movie-lab-defense\<本次标签>\` ，文件对话框可直接粘贴完整路径。

```powershell
Set-Location -LiteralPath 'C:\develop\NOTE_UTP\StudyNote\5 Information Systems\Lab'
$labTag = "LAB2-$(Get-Date -Format 'MMdd-HHmmss')-$([guid]::NewGuid().ToString('N').Substring(0,
$labDemoDir = Join-Path (Join-Path $env:TEMP 'movie-lab-defense') $labTag
New-Item -ItemType Directory -Path $labDemoDir -Force | Out-Null

$labSuccess = Get-Content -LiteralPath 'movie-lab/src/main/webapp/examples/import-valid.json' -R
for ($i = 0; $i -lt $labSuccess.movies.Count; $i++) {
    $labSuccess.movies[$i].name = "$labTag-$($i + 1)"
    $labSuccess.movies[$i].director.passportID = "$labTag-D1"
}
$labSuccessPath = Join-Path $labDemoDir '01-success.json'
$labSuccessJson = $labSuccess | ConvertTo-Json -Depth 12
$labSuccessJson | Set-Content -LiteralPath $labSuccessPath -Encoding UTF8

$labRollback = $labSuccessJson | ConvertFrom-Json
$labRollback.movies[0].name = "$labTag-rollback"
$labRollback.movies[0].director.passportID = "$labTag-D2"
$labRollback.movies[0].director.name = "$labTag-rollback-director"
$labRollback.movies[0].director.location.name = "$labTag-rollback-location"
$labRollbackPath = Join-Path $labDemoDir '02-duplicate-last.json'
$labRollback | ConvertTo-Json -Depth 12 | Set-Content -LiteralPath $labRollbackPath -Encoding UT

"UPLOAD 01: $labSuccessPath"
"UPLOAD 02: $labRollbackPath"
"SEARCH FIRST: $($labSuccess.movies[0].name)"
"SEARCH DUPLICATE: $($labSuccess.movies[1].name)"
"SEARCH ROLLED BACK: $($labRollback.movies[0].name)"
```

代码附录

A. 网页上传与人物复用

浏览器创建 multipart 表单，接口读取上传内容，导入服务再解析文件。前后端约定的部件名是 `file`。接口最多多读一个字节来识别超限，成功时返回 `201`。 app.js 的 `import-form` 提交回调 · ImportResource.java 的上传接口

```
const form = new FormData();
form.append('file', file, file.name);
$('import-result').className = '';
$('import-submit').disabled = true;
bind($('import-result'), 'Импорт выполняется…');
try {
    const result = await api('imports', 'POST', form);
```

```
try (InputStream stream = file.getInputStream()) {
    byte[] bytes = stream.readNBytes(ImportParser.maxBytes() + 1);
    return Response.status(201).entity(imports.upload(user(), file.getSubmittedFileName(), bytes
} finally {
    file.delete();
}
```

导入时先按护照号查本批缓存，再查数据库。 `samePerson` 检查资料是否一致，避免同一护照号指向两套人物资料。找到匹配的人物后复用其数据库 ID。 ImportService.java 的 `person()`

```
private Person person(EntityManager em, Person input, Map<String, Person> cache) {
    if (input == null) return null;
    Person existing = cache.get(input.passportID);
    if (existing == null)
        existing = em.createQuery("SELECT p FROM Person p WHERE p.passportID=:passport", Person
                .setParameter("passport", input.passportID).getResultStream().findFirst().orElse
    if (existing != null) {
        if (!samePerson(input, existing))
            throw new Problem(409, "PERSON_DATA_CONFLICT", "passportID: данные человека отличаю
        cache.put(input.passportID, existing);
        return existing;
    }
    if (input.location != null) em.persist(input.location);
    em.persist(input);
    cache.put(input.passportID, input);
    return input;
}
```

B. 批量事务与回滚

第一段短事务创建历史条目并返回操作 ID，初始状态为 `RUNNING` 。随后 `db.write` 包住整个循环。 `SUCCESS` 和电影数据在同一个主事务提交。确认失败后，独立短事务保存 `FAILED` ，新增数量为

null 。 ImportService.java 的 upload() · ImportOperation.java 的初始状态

```
long operation = db.metadata(em -> {
    ImportOperation op = new ImportOperation();
    op.user = ImportHistoryService.account(em, user);
    op.filename = name;
    em.persist(op);
    em.flush();
    return op.id;
});


return db.write(em -> {
    // Reparse on each confirmed serialization rollback: no managed entities/IDs survive retries
    List<Movie> rows = new ImportParser().parse(bytes).movies();
    Map<String, Person> people = new HashMap<>();
    for (int i = 0; i < rows.size(); i++) {
        Movie m = rows.get(i);
        try {
            m.director = personAt(em, m.director, people, i, "director");
            m.screenwriter = personAt(em, m.screenwriter, people, i, "screenwriter");
            m.operator = personAt(em, m.operator, people, i, "operator");
            em.persist(m.coordinates);
            movies.saveImported(em, m);
        } catch (Problem p) {
            if (p.recordNumber != null) throw p;
            String field = p.code.equals("PERSON_DATA_CONFLICT") ? "person" : "director";
            throw p.at(i + 1, "movies[" + i + "]." + field);
        }
    }
    ImportOperation op = em.find(ImportOperation.class, operation, LockModeType.PESSIMISTIC_WRI
    if (op.status != ImportStatus.RUNNING)
        throw new IllegalStateException("Unexpected import terminal status");
    op.status = ImportStatus.SUCCESS;
    op.addedCount = rows.size();
    op.finishedAt = Instant.now();
    em.flush();
    return ImportHistoryService.dto(op);
});
```

```
db.metadata(em -> {
    ImportOperation op = em.find(ImportOperation.class, operation, LockModeType.PESSIMISTIC_WRI
    if (op.status == ImportStatus.RUNNING) {
        op.status = ImportStatus.FAILED;
        op.addedCount = null;
        op.finishedAt = Instant.now();
        op.errorCode = failure.code;
        op.errorMessage = failure.getMessage().substring(0, Math.min(1000, failure.getMessage()
    }
    return null;
});
```

`saveImported()` 使用循环传进来的同一个 `EntityManager` 。 `flush()` 同步 SQL，事务到外层 `commit()` 才提交。第二条抛出异常时，附录 E 的 `tx.rollback()` 把第一条及本批关联对象一并回滚。MovieService.java 的 `saveImported()`

```
public Movie saveImported(EntityManager em, Movie movie) {
    validate(movie);
    MovieUniquenessValidator.check(em, movie);
    em.persist(movie);
    em.flush();
    return movie;
}
```

C. 字段校验与错误路径

`@Positive` 要求时长为正数， `@NotNull` 要求有值。解析器调用相同的领域校验，并在异常上加上 `movies[1].length` 这样的路径。主写事务之前的 `new ImportParser().parse(bytes)` 会先校验整个文件。 Movie.java 的 `length` · ImportParser.java 的 `MovieService.validate(result)` · 字段校验记录

```
@NotNull
@Positive
@Column(nullable = false)
public Integer length;
```

```
try {
    MovieService.validate(result);
} catch (Problem p) {
    throw p.at(row, path + "." + p.path);
}
```

D. 历史字段、权限和自动刷新

DTO 返回操作 ID、用户名和状态。 `addedCount` 只有在 `SUCCESS` 时才有值，前端失败行用横线。 ImportHistoryService.java 的 `dto()` · app.js 的 `refreshHistory()`

```
row.put("id", op.id.toString());
row.put("operationId", op.id.toString());
row.put("userId", op.user.id.toString());
row.put("user", op.user.username);
row.put("status", op.status.name());
row.put("addedCount", op.status == ImportStatus.SUCCESS ? op.addedCount : null);
row.put("startedAt", op.startedAt.toString());
row.put("finishedAt", op.finishedAt == null ? null : op.finishedAt.toString());
row.put("filename", op.filename);
row.put("errorCode", op.errorCode);
row.put("errorMessage", op.errorMessage);
```

```
const tr = node('tr'), status = node('td');
const [label, variant] = importStatuses[op.status] || [op.status, ''];
status.append(node('span', label, `badge ${variant}`));
tr.append(node('td', op.id), status);
for (const value of [op.user, op.status === 'SUCCESS' ? op.addedCount : '—', op.filename, op.sta
rows.append(tr);
```

列表和分页总数共用所有者条件。普通用户只能查自己的条目。管理员分支省略这个条件。按 ID 请求其他人的条目时，普通用户得到 `404` 。 ImportHistoryService.java 的 `list()` 和 `get()`

```
AppUser actor = account(em, user);
String where = actor.role == UserRole.ADMIN ? "" : " WHERE o.user.id=:user";
var q = em.createQuery("SELECT o FROM ImportOperation o" + where + " ORDER BY o.startedAt DESC,
var count = em.createQuery("SELECT COUNT(o) FROM ImportOperation o" + where, Long.class);
if (actor.role != UserRole.ADMIN) {
    q.setParameter("user", actor.id);
    count.setParameter("user", actor.id);
}
```

```
AppUser actor = account(em, user);
ImportOperation op = em.find(ImportOperation.class, id);
if (op == null || (actor.role != UserRole.ADMIN && !op.user.id.equals(actor.id)))
    throw new Problem(404, "NOT_FOUND", "Операция не найдена");
return dto(op);
```

轮询每两秒触发一次，进入历史分支后直接刷新历史。失败操作没有改动电影集合版本时，历史仍会更新。 app.js 的 `setInterval()`

```
setInterval(async () => {
    if ($('workspace').hidden || state.busy) return;
    state.busy = true;
    try {
        if (state.kind === 'history') await refreshHistory();
        const {revision} = await api('revision');
        if (state.revision !== revision) {
```

E. 组合唯一性、隔离和版本

校验器按导演筛出候选，再比较精确名称与导演 ID。 `IS NULL` 让没有导演的电影也参与检查。编辑时跳过当前记录。普通保存和批量导入共用这段逻辑。 MovieUniquenessValidator.java 的 `check()` · MovieService.java 的 `saveInTransaction()` 与 `saveImported()`

```
public static void check(EntityManager em, Movie movie) {
    MovieKey key = new MovieKey(movie.name, movie.director == null ? null : movie.director.id);
    String director = movie.director == null ? "m.director IS NULL" : "m.director.id=:director";
    var query = em.createQuery("SELECT m FROM Movie m WHERE " + director, Movie.class);
    if (movie.director != null) query.setParameter("director", movie.director.id);
    // Java String.equals preserves exact comparison even under nondeterministic DB collation.
    boolean duplicate = query.getResultStream().anyMatch(m -> !java.util.Objects.equals(m.id, mo
            && key.equals(new MovieKey(m.name, m.director == null ? null : m.director.id)));
    if (duplicate)
        throw new Problem(409, "DUPLICATE_MOVIE", "Название и режиссёр: такая комбинация уже су
}

public record MovieKey(String name, Long directorIdOrNull) {
}
```

先用 `PESSIMISTIC_WRITE` 锁住预先存在的 `AppState` 行，再执行 `work.apply(em)` 里的查重和写入。锁一直持有到提交或回滚。 `READ COMMITTED` 下，后一个拿到锁的事务在查重时能看到前一次提交。应用中的业务写入会因此排队。 Database.java 的 `once()` · Helios 实测隔离级别

```
private <T> T once(Function<EntityManager, T> work, boolean domain, String isolation) {
    try (EntityManager em = factory.createEntityManager()) {
        EntityTransaction tx = em.getTransaction();
        tx.begin();
        try {
            em.createNativeQuery("SET TRANSACTION ISOLATION LEVEL " + (domain ? isolation : "REA
            int timeout = Integer.parseInt(config("MOVIE_LOCK_TIMEOUT_MS", "10000"));
            if (timeout < 1) throw new IllegalArgumentException("lock timeout must be positive"
            em.createNativeQuery("SET LOCAL lock_timeout = '" + timeout + "ms'").executeUpdate(
            AppState state = domain ? em.find(AppState.class, 1L, LockModeType.PESSIMISTIC_WRITE
            if (domain && state == null) throw new IllegalStateException("Сначала выполните data
            T result = work.apply(em);
            if (domain) state.revision++;
            em.flush();
            tx.commit();
            return result;
        } catch (RuntimeException e) {
            if (tx.isActive()) {
                try {
                    tx.rollback();
                } catch (RuntimeException rollback) {
                    e.addSuppressed(rollback);
                }
            }
            throw e;
        }
    }
}
```

编辑和删除在事务中比较客户端保存的版本。 `@Version` 则由 JPA 管理实体版本。编辑先提交时，旧版本的删除得到 `409` 。删除先提交时，后到的编辑找不到电影，得到 `404` 。 MovieService.java · Movie.java

```
Object expected = input.get("version");
if (expected == null || Long.parseLong(expected.toString()) != t.getField("version").getLong(ent
    throw new Problem(409, "Объект изменён другим пользователем. Откройте форму заново.");
```

```
public int delete(String kind, long id, long version) {
    return db.write(em -> {
        Object entity = required(em, type(kind), id);
        try {
            if (type(kind).getField("version").getLong(entity) != version)
                throw new Problem(409, "Объект изменён. Обновите данные перед удалением.");
        } catch (ReflectiveOperationException e) {
            throw new IllegalStateException(e);
        }
```

F. 直接读 JMX 的 XML 和 Helios 验证代码

以下 XML 片段均来自同一个文件。文件内每个测试元素后面的 `hashTree` 装着它的子元素。例如，第 140 行的 `hashTree` 把定时器放在 `Bounded rendezvous` 采样器下面。 lab2-concurrency.jmx

F1. 线程组与内外两层循环（第 12–19、80–82 行）

```
<ThreadGroup guiclass="ThreadGroupGui" testclass="ThreadGroup" testname="${__P(scenario,C02)}">
  <stringProp name="ThreadGroup.num_threads">${__P(threads,2)}</stringProp>
  <intProp name="ThreadGroup.ramp_time">1</intProp>
  <boolProp name="ThreadGroup.same_user_on_next_iteration">true</boolProp>
  <stringProp name="ThreadGroup.on_sample_error">stoptest</stringProp>
  <elementProp name="ThreadGroup.main_controller" elementType="LoopController" guiclass="LoopCor
    <stringProp name="LoopController.loops">1</stringProp>
    <boolProp name="LoopController.continue_forever">false</boolProp>
  </elementProp>
```

```
<LoopController guiclass="LoopControlPanel" testclass="LoopController" testname="Fresh target pe
  <stringProp name="LoopController.loops">${__P(rounds,20)}</stringProp>
  <boolProp name="LoopController.continue_forever">false</boolProp>
```

- `${__P(threads,2)}` 从启动属性读线程数， `2` 是没有传参时的默认值。Python 运行器对每个场景调用一次 JMeter，并传入 `-Jscenario` 、 `-Jthreads` 、 `-Jrounds` 。归档的 C01、C02、C03 分别是 2、10、2 个线程，每组 20 轮，所以目标请求数分别是 40、200、40。 JMeter 属性函数 · C01 命令 · C02 命令 · C03 命令
- `ramp_time=1` 在一秒内启动该组线程。第 18 行的外层循环是 1，登录在内层循环之前，每个线程只登录一次。第 81 行的内层循环才决定重复多少轮业务请求。 `stoptest` 用于样本被判定失败的情况，预期冲突由 F5 里的脚本处理。

F2. 登录、Cookie 和 CSRF（第 23–25、43–51、71–73 行）

```
<CookieManager guiclass="CookiePanel" testclass="CookieManager" testname="Independent session pe
  <collectionProp name="CookieManager.cookies"/>
  <boolProp name="CookieManager.clearEachIteration">false</boolProp>


<stringProp name="HTTPSampler.path">${__P(basePath,/movie-lab/api)}/auth</stringProp>
<stringProp name="HTTPSampler.method">POST</stringProp>


<stringProp name="filename">jmeter/login.groovy</stringProp>
```

`init.groovy` 为线程选测试账号，把登录请求体放进 `${loginBody}`。`POST /auth` 后，Cookie Manager 为每个线程保留独立的会话 Cookie，`login.groovy` 从响应提取 `csrf`，后续写请求用 `X-CSRF-Token` 请求头发送。C02 的 10 指并发线程数。线程会循环复用测试账号，但各自登录，保留各自的会话。 登录初始化 · CSRF 提取 · JMeter Cookie Manager

F3. 每轮取数据、读版本和线程会合（第 85–106、123–144 行）

```
<stringProp name="filename">jmeter/prepare.groovy</stringProp>


<stringProp name="IfController.condition">${hasRead}</stringProp>


<stringProp name="filename">jmeter/read.groovy</stringProp>


<JSR223Sampler guiclass="TestBeanGUI" testclass="JSR223Sampler" testname="Bounded rendezvous" en
  <stringProp name="scriptLanguage">groovy</stringProp>
  <stringProp name="filename">jmeter/barrier.groovy</stringProp>
  <stringProp name="cacheKey">true</stringProp>
  <stringProp name="parameters"></stringProp>
  <stringProp name="script"></stringProp>
</JSR223Sampler>
<hashTree>
  <SyncTimer guiclass="TestBeanGUI" testclass="SyncTimer" testname="Synchronizing Timer">
    <stringProp name="groupSize">${__P(threads,2)}</stringProp>
    <longProp name="timeoutInMs">20000</longProp>
  </SyncTimer>
```

- `prepare.groovy` 从 Python 事先生成的 `fixture.json` 取 `rounds[round].actions[线程编号]`，把 HTTP 方法、路径或上传文件设为当前线程变量。C01 的操作有 `read` 字段，`hasRead` 为真，先 GET 同一部电影，再由 `read.groovy` 检查两边拿到的是同一个初始版本。 每轮参数 · 版本检查
- XML 的 `SyncTimer` 在进入会合采样器前把线程聚到一起，组大小等于线程数，最多等 `20000` 毫秒。`barrier.groovy` 还会按轮次等待一个 `CyclicBarrier`。如果同一轮有人没到，Groovy 的等待会超时报错。线程会合之后才走下面的 HTTP 写请求。 JMeter Synchronizing Timer · Groovy 屏障

F4. 普通写请求和导入请求（第 148、159–167、198、209–218 行）

```
<stringProp name="IfController.condition">${isCrud}</stringProp>


<stringProp name="HTTPSampler.path">${__P(basePath,/movie-lab/api)}/${targetPath}</stringProp>
<stringProp name="HTTPSampler.method">${targetMethod}</stringProp>


<stringProp name="IfController.condition">${isUpload}</stringProp>
```

`prepare.groovy` 根据操作有没有文件来设置两个条件。C01、C02 走 `isCrud`，复用同一个采样器。C01 的两个线程分别用 `PUT` 和 `DELETE`，C02 用 `POST` 创建电影。C03 走 `isUpload`，把 `${uploadFile}` 对应的 JSON 文件作为名为 `file` 的 multipart 部件发给 `POST /imports`，与网页用的是同一接口。

```
<stringProp name="HTTPSampler.path">${__P(basePath,/movie-lab/api)}/imports</stringProp>
<stringProp name="HTTPSampler.method">POST</stringProp>
<boolProp name="HTTPSampler.use_keepalive">true</boolProp>
<boolProp name="HTTPSampler.DO_MULTIPART_POST">true</boolProp>
<elementProp name="HTTPsampler.Files" elementType="HTTPFileArgs">
  <collectionProp name="HTTPFileArgs.files">
    <elementProp name="${uploadFile}" elementType="HTTPFileArg">
      <stringProp name="File.path">${uploadFile}</stringProp>
      <stringProp name="File.paramname">file</stringProp>
      <stringProp name="File.mimetype">application/json</stringProp>
    </elementProp>
  </collectionProp>
</elementProp>
```

F5. JMX 的后置处理器怎样对应结果（第 187–189、237–239 行）

```
<JSR223PostProcessor guiclass="TestBeanGUI" testclass="JSR223PostProcessor" testname="Business s
  <stringProp name="scriptLanguage">groovy</stringProp>
  <stringProp name="filename">jmeter/record.groovy</stringProp>
```

两条 HTTP 分支都挂了这个后置处理器。 `record.groovy` 用当前操作的预期状态码判定样本，C03 的 `409` 还必须是 `DUPLICATE_MOVIE` 且路径为 `movies[1].director` 。脚本会把实际状态码与 `passed` 分开写入 `responses.jsonl` ，所以可以同时看到 `status: 409` 和 `passed: true` 。Python 再核对每轮只有一个获胜操作和数据库最终状态。 record.groovy · C03 原始响应

```
boolean valid = action.expected.contains(status)
if (status == 409 && action.code)
    valid = valid && data.code == action.code
if (status == 409 && action.errorPath)
    valid = valid && data.path == action.errorPath

prev.setSuccessful(valid)
```

`prepare()` 为 C01 构造同一电影版本上的编辑和删除，为 C02 给所有线程相同的名称与导演 ID，为 C03 构造首条各异、末条相同的两个 JSON。 lab2-jmeter.py

```
if scenario == 'C01':
    target = api.create('movies', movie(prefix + 'old', coordinates))
    path, version = f"movies/{target['id']}", target['version']
    meta['target'], meta['targetVersion'] = target['id'], version
    meta['actions'] = [
        {'method': 'PUT', 'path': path, 'read': path, 'version': version,
         'body': {'tagline': prefix + 'updated', 'version': version}, 'expected': [200, 404]},
        {'method': 'DELETE', 'path': f'{path}?version={version}', 'read': path,
         'version': version, 'expected': [200, 409]},
    ]
elif scenario == 'C02':
    action = {'method': 'POST', 'path': 'movies', 'body': movie(prefix + 'same', coordinates, d
              'expected': [201, 409], 'code': 'DUPLICATE_MOVIE'}
    meta['actions'] = [action] * threads
else:
    for i in range(threads):
        names = [f'{prefix}unique{i}', prefix + 'shared']
        document = {'formatVersion': 1,
                    'movies': [import_movie(name, prefix + 'passport') for name in names]}
        file = out / f'r{r}-p{i}.json'
        file.write_text(json.dumps(document), encoding='utf-8')
        meta['actions'].append({
            'method': 'POST', 'path': 'imports', 'file': str(file.resolve()),
            'expected': [201, 409], 'code': 'DUPLICATE_MOVIE',
            'errorPath': 'movies[1].director',
        })
```

`verify()` 在 C01 重新查电影，核对标语和版本，或者确认删除。C02 核对数据库只有一个匹配的组合键。C03 用 `Counter` 比较最终电影名和获胜文件，按四张表比较前后增量，并核对两次导入的历史。

lab2-jmeter.py · C01 断言 · C02 断言 · C03 断言

```
    for row, action in zip(rows, actions):
        history = histories.get(row['operationId'])
        success = row['status'] == 201
        check(history is not None and history['filename'] == Path(action['file']).name
              and history['status'] == ('SUCCESS' if success else 'FAILED')
              and history['count'] == (2 if success else None), f'{r}: history differs from uplo
        if success:
            winners['import'] += 1
            expected = json.loads(Path(action['file']).read_text(encoding='utf-8'))['movies']
            check(collections.Counter(m['name'] for m in movies) ==
                  collections.Counter(m['name'] for m in expected), f'{r}: winning file differs
        else:
            late_duplicate = (row['status'] == 409 and row['code'] == 'DUPLICATE_MOVIE'
                              and row['errorPath'] == 'movies[1].director')
            check(late_duplicate, f'{r}: rejection must occur on the second movie')
            late_conflicts += int(late_duplicate)

domain_delta = {key: after[key] - before[key] for key in ('movies', 'coordinates', 'people', 'lo
if scenario == 'C01':
    expected_delta = {'movies': -winners['delete'], 'coordinates': 0, 'people': 0, 'locations':
elif scenario == 'C02':
    expected_delta = {'movies': rounds, 'coordinates': 0, 'people': 0, 'locations': 0}
else:
    imports = winners['import']
    expected_delta = {'movies': 2 * imports, 'coordinates': 2 * imports, 'people': imports, 'loc
check(domain_delta == expected_delta, f'Unexpected domain delta or failed import residue: {domai
```

现场对照 C03 第一轮时可打开两份输入，指出 `unique0`、`unique1` 和 `shared`。原始结果对应失败操作 `971` 与成功操作 `972`。 线程 0 的 JSON · 线程 1 的 JSON · 原始结果

展示运行来源时，指配置中的 `helios.cs.ifmo.ru`、`movies-helios-test`、`READ COMMITTED` 和实际命令参数。HTML 中按 `C03 import` 看目标请求，`Total` 还包含登录、准备数据等辅助请求。 Helios 配置 · C03 运行命令 · JMeter HTML · JMeter 运行说明